

COOPERATIVE ROBOTICS

Authors:

Penna Lorenzo (4953984)

Vercellino Alessandro (4843829)

Date: 25/01/2026

General notes

- Exercise 1 is done with the ROBUST matlab main and unity visualization tools. Exercises 2-3 are done with the Franka simulation tools.
- Comment and discuss the simulations, in a concise scientific manner. Further comments, other than the questions, can be added, although there is no need to write 10 pages for each exercise.
- Aid the discussion with screenshots of the simulated environment (compress them to maintain a small overall file size), and graphs of the relevant variables (i.e. activation functions of inequality tasks, task variables, and so on). Graphs should always report the units of measure in both axes, and legends whenever relevant.
- You can add vertical bars to plots to highlight periods of time (e.g. when one action is active), or vertical ones to highlight thresholds (e.g. the minimum altitude value).
- Report the thresholds whenever relevant.
- Report the mathematical formula employed to derive the task jacobians and the control laws when asked, including where they are projected.
- If needed, part of the code can be inserted as a discussion reference.
- Provide your answers in the subsection where the question is made.

Use the following template when you need to discuss the hierarchy of tasks of a given action or set of actions. Please report the tasks in the same order as in the unified hierarchy implemented in your code.

Table 1: Example of actions/hierarchy table: a number in a given cell represents the priority of the control task (row) in the hierarchy of the control action (column). The type column indicates whether the objective is an equality (E) or inequality (I) one.

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
Task name A	I	1		1
Task name B	I	2	1	
Task name C	E		2	2

1 Exercise 1: Implement a Complete Mission

Implement several actions to reach a point, land, and then perform the manipulation of a target object.

Initialize the vehicle in a position that allows you to test all the tasks. Then, use a "safe waypoint navigation action" to reach the following position:

$$\begin{bmatrix} 10.5 & 37.5 & -38 & 0 & -0.06 & 0.5 \end{bmatrix}^T$$

Then land, aligning to the nodule. In particular, the x axis of the vehicle should align to the projection, on the inertial horizontal plane, of the unit vector joining the vehicle frame to the nodule frame. Once landed, implement a "fixed-based manipulation action" to reach the target nodule (mimicking the scanning of the nodule). During this manipulation phase, the vehicle should not move for any reason. Considering that the goal position for the vehicle could be not so precise with respect to the position of the nodule, implement a solution that guarantees that, once landed, the nodule is certainly within the manipulator's workspace. If needed, you can change also the vehicle's goal position if you need to stress the system and force certain tasks to activate (e.g., the distance to the nodule one).

1.1 Q1: Report the unified hierarchy of tasks used, the actions and their tasks, specifying their priorities in each action.

The UVMS mission is controlled using a task-priority inverse kinematics framework (iCAT). The system relies on a `unified_task_list` to establish a strict global priority order. Tasks appearing earlier in this list are prioritized over subsequent ones, which are solved in the null-space of the higher-priority tasks.

Hierarchy and Action Mapping

The following table details the hierarchy of control tasks for the three defined actions: **Navigation** ($\mathcal{A}_1$), **Landing** ($\mathcal{A}_2$), and **Manipulation** ($\mathcal{A}_3$). The priority numbers reflect the execution order derived from the unified list (1 being the highest priority).

The *Type* column categorizes tasks as either Equality objectives (E), which must always be satisfied (e.g., tracking), or Inequality constraints (I), which typically activate only when safety thresholds are breached.

Table 2: Hierarchy of control tasks: a number represents the priority of the task within the specific action. Type: E (Equality), I (Inequality).

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
TaskAltitude	I	1		
TaskStopVehicle	E			1
TaskHorizontalAttitude	E		1	
TaskAlign	E		2	
TaskLand	E		3	
TaskOrientation	E	2		
TaskPosition	E	3	4	
TaskProximityBoundary	I		5	
TaskTool	E			2

Task Description

Each task corresponds to a specific control objective implemented in the system:

- **TaskAltitude (I):** A safety constraint that ensures the vehicle maintains a safe distance from the seabed. It uses a decreasing bell-shaped activation function, effectively acting as an inequality task that becomes active only when the altitude drops below a safety threshold.

- **TaskStopVehicle (E):** Forces the linear and angular velocities of the vehicle to zero. This creates a stable floating base for the manipulator during high-precision operations.
- **TaskHorizontalAttitude (E):** Regulates the vehicle’s roll and pitch angles to zero. This is crucial during the landing phase to ensure the vehicle settles flat on the ground and maintains sensor stability.
- **TaskAlign (E):** A guidance task that controls the vehicle’s yaw (heading). It aligns the vehicle’s forward axis (X) toward the target nodule projected on the horizontal plane, ensuring the target is in front of the robot before landing.
- **TaskLand (E):** Controls the vertical descent velocity. Unlike general positioning, this task specifically manages the approach to the landing altitude target.
- **TaskOrientation (E):** Manages the full 3D orientation (roll, pitch, yaw) of the vehicle during the navigation phase to follow the desired trajectory.
- **TaskPosition (E):** Controls the Cartesian position (x, y) of the vehicle in the world frame. It is responsible for driving the robot from the starting point to the intervention area.
- **TaskProximityBoundary (I):** Acts as a ”virtual fence” or soft inequality constraint. It monitors the horizontal distance to the target; if the vehicle drifts beyond a specific radius (e.g., 1.5m), it generates a velocity command to pull the vehicle back, ensuring the nodule remains within the arm’s reachable workspace.
- **TaskTool (E):** Controls the position and orientation of the manipulator’s end-effector. Being the lowest priority task, it exploits the remaining degrees of freedom to perform the manipulation on the nodule.

1.2 Q2: Comment the behaviour of the robot, supported by relevant plots. For example, show the altitude and the altitude activation to show that when the altitude becomes too small, the tasks gets activated and a minimum value is guaranteed.

The simulation results illustrate the correct execution of the mission phases and the effectiveness of the safety constraints implemented in the Task-Priority architecture.

The Cartesian and Joint velocity profiles recorded by the `SimulationLogger` clearly distinguish the three operational phases defined in the mission controller:

- **Navigation Phase ($t < t_{nav}$):** The vehicle exhibits significant planar linear velocities ($\mathbf{v}_{\nu,xy}$) as it approaches the target coordinates defined in `RobustMain.m`. Regarding angular velocities, the vehicle’s yaw rate ($\omega_{\nu,z}$) is active to align the heading with the target, driven by `TaskOrientation`, while roll and pitch rates are minimized. The arm joint velocities ($\dot{\mathbf{q}}$) remain low as the manipulator is essentially holding its configuration during transport.
- **Landing Phase ($t_{nav} < t < t_{land}$):** Upon switching to the landing action, the vertical velocity component of the vehicle ($v_{\nu,z}$) becomes dominant, driven by `TaskLand`. Simultaneously, the angular velocities show the effect of `TaskHorizontalAttitude`, which actively regulates the vehicle’s roll and pitch rates ($\omega_{\nu,x}, \omega_{\nu,y}$) to zero to maintain a flat descent profile. The heading alignment continues via `TaskAlign`.
- **Manipulation Phase ($t > t_{land}$):** Once the landing is complete, the vehicle velocities ($\mathbf{v}_{\nu}$) drop to near zero due to the activation of `TaskStopVehicle`. In contrast, the tool velocities ($\dot{\mathbf{x}}_{tool}$) show significant activity in both linear and angular components. The angular components (ω_{tool}) are particularly relevant here, as `TaskTool` drives the end-effector to achieve the desired orientation ($\omega_{arm-goal}$) relative to the nodule, requiring coordinated motion of the arm joints ($\dot{\mathbf{q}}$).

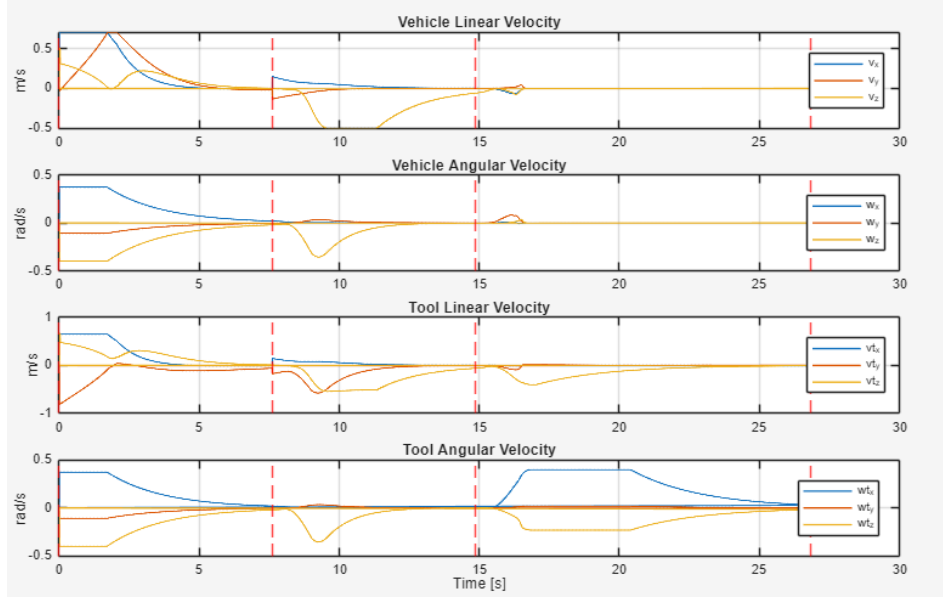


Figure 1: vehicle and tool velocities

1.3 Safety Critical Behavior: Altitude Task

The robustness of the control architecture is demonstrated by the activation of the safety constraint encoded in **TaskAltitude**. This task prevents the vehicle from colliding with the seabed.

The task relies on a soft-activation mechanism governed by a decreasing bell-shaped function, defined as follows:

$$A(h) = \text{DecreasingBellShaped}(h_{start}, h_{end}, 0, 1, h(t)) \quad (1)$$

where $h_{start} = 2.0\text{ m}$ and $h_{end} = 3.0\text{ m}$. The behavior observed in the altitude plots can be analyzed in three stages:

1. **Descent:** Initially, while the altitude $h(t) > 3.0\text{ m}$, the activation $A(h)$ remains zero, allowing the lower-priority **TaskLand** to drive the vehicle downwards without interference.
2. **Activation:** As the vehicle breaches the safety threshold ($h(t) < 3.0\text{ m}$), the activation function $A(h)$ smoothly increases from 0.
3. **Regulation:** The active task injects a repulsive velocity command $\dot{x} = -k_p(h(t) - h_{min})$ into the iCAT stack. This creates a “virtual floor,” effectively overriding the landing command to maintain a safe minimum distance from the seabed.

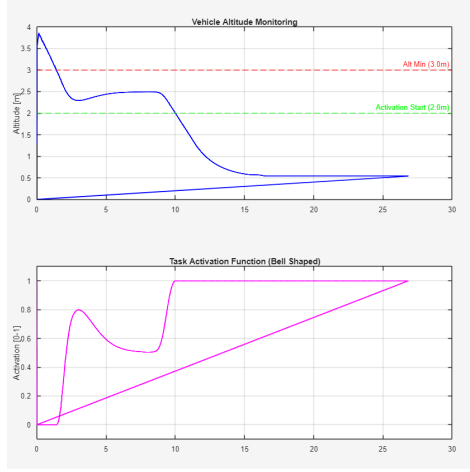


Figure 2: minimum altitude

This mechanism guarantees that the safety of the UVMS is prioritized over mission execution when environmental limits are approached.

1.4 Safety Soft-Constraint: Horizontal Attitude Task

In addition to the absolute altitude safety, the simulation analysis highlights the role of the `TaskHorizontalAttitude` in maintaining vehicle stability.

Unlike the altitude task, which enforces a "hard" floor, this task acts as a soft constraint on the vehicle's orientation (specifically the tilt angle θ between the vehicle z -axis and the global vertical). The relevant plots show:

- **Tilt Angle Dynamics:** The vehicle's tilt angle $\theta(t)$ fluctuates during the navigation and landing phases due to the coupling with translation maneuvers.
- **Selective Activation:** The activation function is defined as:

$$A(\theta) = \text{IncreasingBellShaped}(0.1, 0.2, 0, 1, \theta) \quad (2)$$

This implies that for small perturbations ($\theta < 0.1$ rad), the task remains inactive ($A = 0$), allowing the vehicle maximum agility.

- **Correction:** As seen in the plots, when θ approaches the soft threshold of 0.1 rad, the activation rises, injecting a corrective torque that prevents the tilt from reaching the critical limit of 0.2 rad. This confirms the system's ability to balance performance (speed) with stability.

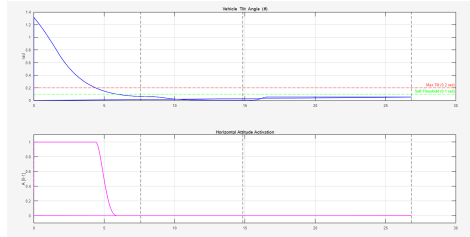


Figure 3: minimum altitude

1.5 Q3: What is the Jacobian relationship for the Alignment to Target control task? How was the task reference computed?

The `TaskAlign` task ensures the vehicle's forward axis (X_v) is aligned with the vector pointing toward the target (nodule), projected on the horizontal plane. This implementation uses a geometric approach based on vector alignment rather than simple Euler angle differences.

Jacobian Relationship

The Jacobian $J_{align} \in \mathbb{R}^{1 \times 13}$ for this task is defined by projecting the vehicle's angular velocity onto the required rotation axis. According to the source code:

$$J_{align} = \mathbf{n}^T \cdot \begin{bmatrix} \mathbf{0}_{3 \times 7} & \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \end{bmatrix} \quad (3)$$

Where:

- $\mathbf{n}$ is the unit vector representing the rotation axis, computed as the normalized cross product between the current heading $\mathbf{u}_{curr}$ and the desired direction $\mathbf{u}_{des}$ ($\mathbf{n} = \frac{\mathbf{u}_{curr} \times \mathbf{u}_{des}}{\|\mathbf{u}_{curr} \times \mathbf{u}_{des}\|}$).
- $\mathbf{0}_{3 \times 7}$ represents the decoupling from the manipulator's joints.
- $\mathbf{I}_{3 \times 3}$ maps the vehicle's angular velocities (ω_v) in the global frame.

This Jacobian effectively extracts the component of the vehicle's rotation that contributes to reducing the alignment error.

Task Reference Computation

The task reference $\bar{x}$ is computed as a scalar angular velocity using a geometric approach:

1. Direction Vectors:

- Current heading: $\mathbf{u}_{curr}$ is extracted from the first column of the vehicle's rotation matrix (`robot.wTv(1:3, 1)`).
- Desired direction: $\mathbf{v}_{des} = \mathbf{p}_{nodule} - \mathbf{p}_{vehicle}$. The vertical component is set to zero ($\mathbf{v}_{des}(3) = 0$) to focus solely on horizontal alignment.

2. Error Angle: Instead of using `atan2` on coordinates, the error is computed using the dot and cross products between the two unit vectors:

$$e_{angle} = \text{atan2}(\|\mathbf{u}_{des} \times \mathbf{u}_{curr}\|, \mathbf{u}_{des} \cdot \mathbf{u}_{curr}) \quad (4)$$

3. Control Law: The reference velocity is generated through a proportional controller with saturation:

$$\bar{x} = \text{Saturate}(K_p \cdot e_{angle}, \text{max_vel}) \quad (5)$$

where $K_p = 1.0$ and the velocity is capped at 0.5 rad/s.

This formulation is numerically robust as it handles the singularity when the distance to the target is too small (setting $\mathbf{u}_{des} = \mathbf{u}_{curr}$), preventing undefined behaviors.

1.6 Q4: What have you done to guarantee that the nodule is within the manipulator's workspace?

To ensure that the target (nodule) remains within the manipulator's workspace, the control system implements a two-fold strategy: a high-level mission goal correction and a low-level kinematic task named `TaskProximityBoundary`.

Autonomous Goal Correction

During the *Landing* phase, the mission controller monitors the horizontal distance $dist_target$ between the vehicle and the nodule. If $dist_target > r_{max}$ (where $r_{max} = 1.3$ m), the system triggers an automatic repositioning:

1. **Directional Vector:** The system computes $\mathbf{u}_{dir} = \frac{\mathbf{p}_{nodule} - \mathbf{p}_{vehicle}}{\|\mathbf{p}_{nodule} - \mathbf{p}_{vehicle}\|}$ projected on the XY plane.
2. **Excess Distance:** The distance outside the workspace is calculated as $excess = dist_target - r_{max}$.
3. **Goal Update:** The vehicle's target position is shifted: $\mathbf{pos}_{goal} = \mathbf{pos}_{goal} + (\mathbf{u}_{dir} \cdot excess)$.

Kinematic Task: TaskProximityBoundary

While the goal correction provides the target coordinates, the **TaskProximityBoundary** acts as a safety constraint within the iCAT framework to keep the vehicle within a reachable range (1.2 to 1.5 meters) from the nodule.

Jacobian Relationship

The Jacobian $J_{prox} \in \mathbb{R}^{2 \times 13}$ relates the horizontal error between the vehicle and the target to the system velocities. It is defined as:

$$J_{prox} = [\mathbf{0}_{2 \times 7} \quad -^w\mathbf{R}_v(1:2, 1:3) \quad \mathbf{0}_{2 \times 3}] \quad (6)$$

This Jacobian maps the vehicle's linear velocities directly to the task space, while joints velocities have no influence ($\mathbf{0}_{2 \times 7}$).

Task Reference and Activation

The task computes a restorative velocity to pull the vehicle back toward the target if it drifts away:

- **Reference:** $\bar{x} = \text{Saturate}(-0.4 \cdot \mathbf{v}_{lin,xy}, 0.2)$. It generates a velocity vector opposite to the position error.
- **Activation:** The task uses a transition function to activate only when the vehicle is at the edge of the workspace:

$$\mathbf{A} = \mathbf{I}_{2 \times 2} \cdot \text{IncreasingBellShapedFunction}(1.3, 1.7, 0, 1, dist) \quad (7)$$

The task is inactive ($A = 0$) when the vehicle is close ($< 1.5\text{m}$) and becomes fully active ($A = 1$) when the distance exceeds 1.7m.

Transition to Manipulation

The mission only enters the *Manipulation* phase once the vehicle is stabilized at the corrected goal (`task_position.error` < 0.2) and the landing altitude is reached. This sequence ensures the arm always starts with the nodule perfectly reachable.

2 Exercise 2: Bimanual manipulation

In this exercise it is required to perform a bimanual manipulation by implementing the task priority algorithm considering two manipulators as a single robot. The manipulator adopted for this assignment is the Franka Panda by Emika (see Figure 4)). A simulation in python is provided, in order to visualize the two robots and test your Matlab implementation.

1. The transformations between the world and the base of each robot must be computed knowing that:
 - (a) The left arm base coincides with the world frame.
 - (b) The right arm base is rotated of π w.r.t. z-axis and is positioned 1.06 m along the x-axis and -0.01 m along the y-axis.
2. The transformation from each base to the respective end-effector is given in the code and is retrieved from the URDF model thanks to the *Robotic System Toolbox* of Matlab.
3. Then, define the tool frame for both manipulators; the tool frames must be placed at 21.24 cm along the z-axis of the end-effector frame and rotated with an angle of -44.9949 deg around the z-axis of the end-effector.
4. Implement a “move to” action that includes the joint limits task.

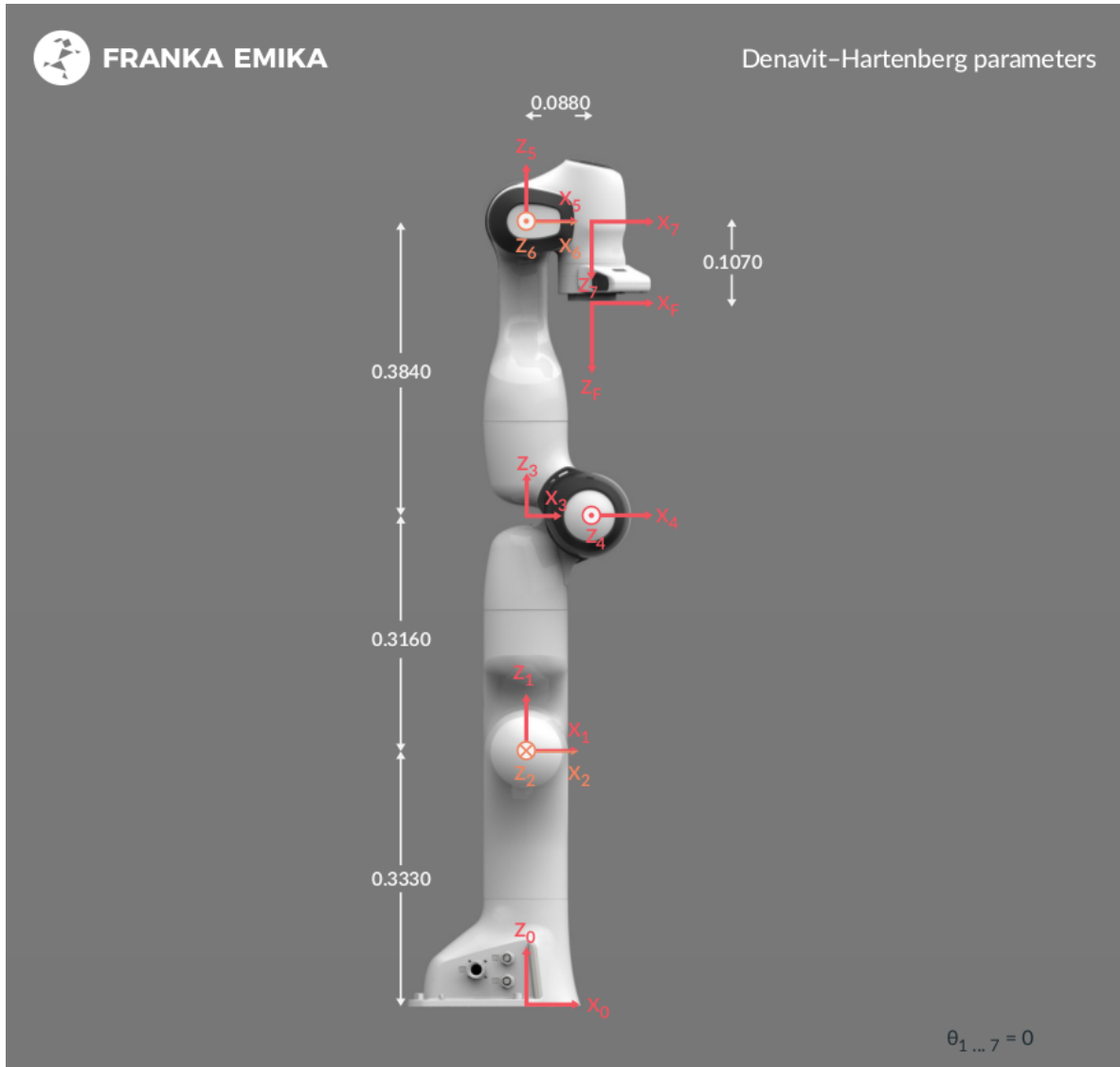


Figure 4: Robot Specifications

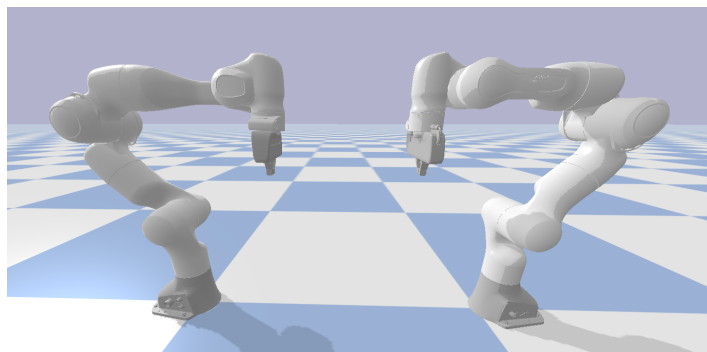


Figure 5: Initial position of the robots

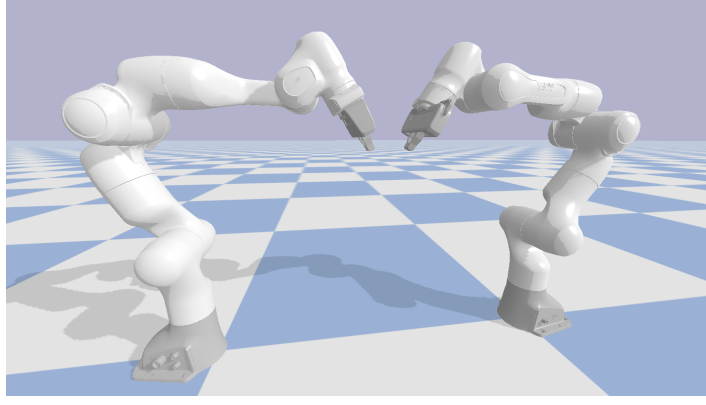


Figure 6: Relative orientation at the grasping position

5. The first phase foresees to move the tool frame of both manipulators to the grasping points, by implementing a “move to” action and its corresponding tasks. Define the goal frames for each manipulator such that their origin corresponds to the grasping points. **HINT:** the position of the grasping points can be computed by knowing the origin of the object frame wO_o and the object length l_{obj} . The goal orientation of the tool frames is obtained by rotating the tool frames 30 deg around their y-axis; the manipulators should reach the configuration depicted in Figure 6.

$${}^wO_o = [0.5, 0, 0.59]; \quad (8)$$

$$l_{obj} = 10 \text{ (cm)}. \quad (9)$$

6. During this phase of the mission, the following safety tasks must be implemented: *end-effector minimum altitude* and *joint limits*. The joint limits specifications can be found in the datasheet of the robot. The minimum altitude from the table should be 15 cm.

Once the manipulators reach the grasping points, the second phase of the mission should start. Now, implement the Bimanual Rigid Grasping Constraint task to carry the object as a rigid body.

1. Define the object frame as a rigid body attached to the tool frame of each manipulator. **HINT:** Compute this quantity after reaching the grasping point.
2. Define the rigid grasp task.
3. Then, move the object to another position while both manipulators hold it firmly, e.g. ${}^wO_g = [0.65, -0.35, 0.28]^T (m)$
4. Note that the transition for the *Bimanual Rigid Constraint* should be a binary one, i.e., without smoothness. This is the nature of the constraint, i.e., either it exists or not. Modify the *ActionManager* class to take into account the different nature of this task (a constraint one).

Once the object has been moved to the required position you have to implement a final phase of the mission in which the joint velocities are set to zero, and every action is deactivated except for the minimum altitude task.

Repeat the simulation, using a goal position or by changing the grasping in such a way that one of the two manipulators activates multiple safety tasks, to stress the constraint task.

2.1 Q1: Report the unified hierarchy of tasks used and their priorities in each action.

The control architecture for the bimanual manipulation mission is implemented using an *iCAT* (inequality Control Action based on Task priority) framework. The global priority of all tasks is strictly defined by the `unified_task_list` in the main execution script.

The hierarchy ensures that critical constraints (such as physical consistency and safety limits) are satisfied before operational objectives (such as moving the tools or the object).

Global Unified Task List

The `ActionManager` processes tasks in the following order of decreasing priority (1 = Highest):

1. **RigidConstraint**: Enforces the closed-chain kinematic constraint between the two end-effectors holding the object.
2. **JointLimits (L/R)**: Prevents the robot joints from exceeding their mechanical limits.
3. **ToolAltitudeLimit (L/R)**: Prevents the tools from colliding with the table/ground (safety floor).
4. **ToolTask (L/R)**: Controls the independent motion of the left and right end-effectors (used during approach).
5. **ObjectTask (L/R)**: Controls the cooperative motion of the object once grasped.
6. **StopTask (L/R)**: Forces the joint velocities to zero to safely terminate the mission.

Actions and Priority Assignment

The mission is divided into three distinct phases (Actions), each activating a specific subset of the unified list. The priority of a task within an action is determined by its position in the global unified list.

The actions are defined as follows:

- **Action 1 ($\mathcal{A}_1$): Go To Grasping Point.** The arms move independently to reach the grasp pre-pose.
- **Action 2 ($\mathcal{A}_2$): Bimanual Manipulation.** The arms grasp the object, forming a closed kinematic chain, and move the object cooperatively.
- **Action 3 ($\mathcal{A}_3$): Stop Manipulation.** The system brings all velocities to zero while maintaining safety limits.

Table 3: Hierarchy of control tasks for the Bimanual Manipulation mission. Numbers indicate the priority level (1 = Highest). Type: E (Equality), I (Inequality).

Task Name	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
<code>rigid.constraint</code>	E		1	
<code>j1.L (Joint Limit L)</code>	I	1	2	
<code>j1.R (Joint Limit R)</code>	I	2	3	
<code>tool.alt.task.L</code>	I	3	4	1
<code>tool.alt.task.R</code>	I	4	5	2
<code>left.tool.task</code>	E	5		
<code>right.tool.task</code>	E	6		
<code>object.task.l</code>	E		6	
<code>object.task.r</code>	E		7	
<code>stop.velocities.task.R</code>	E			3
<code>stop.velocities.task.L</code>	E			4

Detailed Task Description

- **RigidConstraint (E)**: This is the highest priority task in the unified list. It describes the physical constraint of two arms holding a single rigid object. It ensures that the relative pose between the left and right end-effectors remains constant, preventing the object from being dropped or the arms from tearing it apart.

- **JointLimitTask (I):** A safety inequality task defined for both the Left (`j1.L`) and Right (`j1.R`) arms. It activates to repel the joints from their physical limits (q_{min}, q_{max}) using a safety buffer (set to 0.15 rad in the code).
- **ToolAltitudeLimit (I):** A safety inequality task (`tool_alt_task.L/R`) that prevents the end-effectors from crashing into the workspace surface. It is active in all phases to guarantee hardware safety.
- **ToolTask (E):** Used exclusively in Phase 1 ($\mathcal{A}_1$). It controls the 6-DoF pose of the end-effectors to align them with the grasping frames defined relative to the object. The right tool task has a lower priority than the left one in the global list, meaning the left arm is prioritized in case of kinematic conflicts during the approach.
- **ObjectTask (E):** Used in Phase 2 ($\mathcal{A}_2$). Instead of controlling the hands directly, this task defines the desired motion of the *Object* itself. The system computes the necessary joint velocities for both arms to move the object to the target pose (**wTog**) while maintaining the grasp.
- **StopTask (E):** Used in Phase 3 ($\mathcal{A}_3$). It acts as a regulator, setting the desired velocity vector to zero. It is placed at the bottom of the hierarchy to ensure that the robot stops only if doing so does not violate higher-priority safety constraints (like altitude limits).

2.2 Q2: What is the Jacobian relationship for the Joint Limits task? How was the task reference computed?

The **JointLimitTask** is designed to keep the manipulator's joints within safe mechanical operating ranges. It acts as a repulsive potential field that activates only when the joints approach their physical limits.

Jacobian Relationship

The Jacobian matrix for the Joint Limits task, $J_{jl} \in \mathbb{R}^{7 \times 14}$, maps the generalized velocity vector of the bimanual system ($\dot{q}_{sys} = [\dot{q}_L^T, \dot{q}_R^T]^T$) directly to the joint velocities of the specific arm being controlled. Since the task operates directly in the configuration space of a single arm (7 DoF), the Jacobian is a selection matrix composed of an identity block and a zero block.

For the **Left Arm** (ID = 'L'), the Jacobian selects the first 7 degrees of freedom:

$$J_{jl,L} = [\mathbf{I}_{7 \times 7} \quad \mathbf{0}_{7 \times 7}] \quad (10)$$

For the **Right Arm** (ID = 'R'), the Jacobian selects the last 7 degrees of freedom:

$$J_{jl,R} = [\mathbf{0}_{7 \times 7} \quad \mathbf{I}_{7 \times 7}] \quad (11)$$

Task Reference Computation

The task reference $\bar{x}$ (desired joint velocity) is computed to generate a repulsive velocity that pushes the joints away from the limits if they enter a defined safety buffer δ (set to 0.2 rad).

The computation proceeds as follows:

1. **Error Calculation:** The system calculates the penetration depth into the lower and upper safety buffers:

$$e_{min} = (q_{min} + \delta) - q \quad (12)$$

$$e_{max} = q - (q_{max} - \delta) \quad (13)$$

2. **Repulsive Velocity:** The reference velocity is proportional to the violation magnitude. It pushes in the positive direction if the lower limit is approached ($e_{min} > 0$) and in the negative direction if the upper limit is approached ($e_{max} > 0$):

$$\dot{q}_{ref} = \lambda \cdot (\max(e_{min}, 0) - \max(e_{max}, 0)) \quad (14)$$

where $\lambda = 0.5$ is the proportional gain.

3. **Saturation:** Finally, the computed reference is saturated to ensure safe operation, with a maximum allowed correction velocity of 0.5 rad/s:

$$\ddot{x} = \text{Saturate}(\dot{q}_{ref}, 0.5) \quad (15)$$

This logic ensures that if a joint is within the safe range $[q_{min} + \delta, q_{max} - \delta]$, the reference velocity is zero, and the task does not interfere with the robot's motion.

2.3 Q3: What is the Jacobian relationship for the Bimanual Rigid Grasping Constraint task? How was the task reference computed?

The `RigidConstrTask` enforces the closed-chain kinematic constraint required when two manipulators grasp a single rigid object. It ensures that the two end-effectors move in a coordinated manner, maintaining zero relative velocity between them.

Jacobian Relationship

The Jacobian matrix for this task, $J_{rigid} \in \mathbb{R}^{6 \times 14}$, represents the differential relationship between the combined joint velocities of the system ($\dot{q} = [\dot{q}_L^T, \dot{q}_R^T]^T$) and the relative velocity error between the two grasping points projected onto the object frame.

It is constructed by concatenating the Jacobians of the left and right arms, transformed to the common object frame:

$$J_{rigid} = [J_{o,L} \quad -J_{o,R}] \quad (16)$$

Where $J_{o,\{L,R\}}$ are the geometric Jacobians of the left and right end-effectors projected to the object origin using the twist transformation matrix (Grasp Matrix) X :

$$J_{o,\{L,R\}} = X_{\{L,R\}} \cdot {}^w J_{t,\{L,R\}} \quad (17)$$

The transformation matrix X is defined as:

$$X = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \\ -S(\mathbf{r}) & \mathbf{I}_3 \end{bmatrix} \quad (18)$$

Here, $S(\mathbf{r})$ is the skew-symmetric matrix of the lever arm vector $\mathbf{r}$ connecting the tool frame to the object frame ($\mathbf{r} = \mathbf{p}_{obj} - \mathbf{p}_{tool}$).

Task Reference Computation

The task reference $\ddot{x}$ describes the desired relative velocity between the two projected end-effector frames. Since the object is rigid and the grasp is firm, the left and right arms must not move relative to each other. Therefore, the reference is identically zero:

$$\ddot{x} = \mathbf{0}_{6 \times 1} \quad (19)$$

This forces the solver to find joint velocities $\dot{q}_L$ and $\dot{q}_R$ such that the velocity of the left arm (projected to the object) equals exactly the velocity of the right arm (projected to the object).

2.4 Q4: Comment the behaviour of the robots, using relevant plots. In particular, show the difference (if any) between the desired object velocity, and the velocities of the two end-effectors in the two cases. Add plots to show that the bimanual manipulation unfolds effectively (e.g., the distance between the tool frames is constant to show that the kinematic constraint is satisfied)

The following analysis evaluates the performance of the bimanual control architecture defined in `main.m`. The discussion is structured to compare the nominal cooperative behavior against safety-critical scenarios where specific tasks override the primary motion commands.

1. General Analysis: Standard Cooperative Behavior

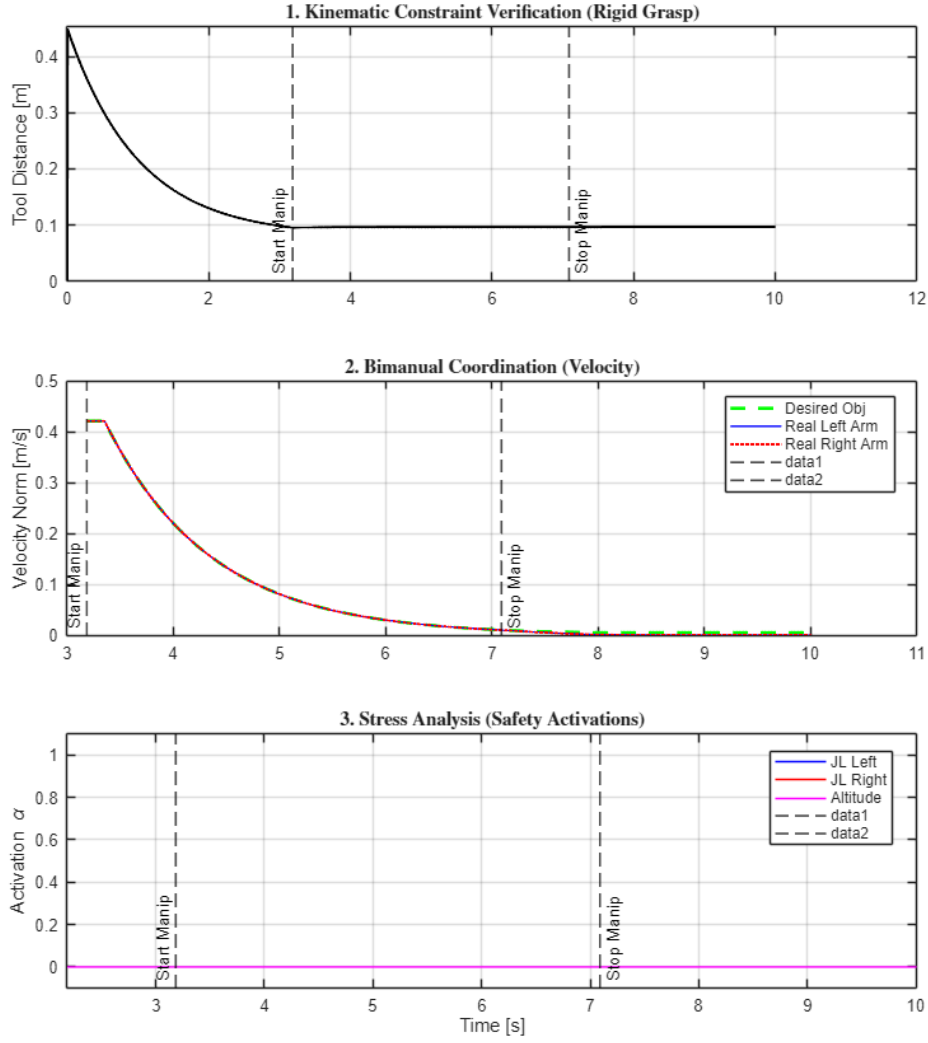


Figure 7: Nominal Behavior: The end-effectors (Red/Blue) perfectly track the desired object velocity (Green).

In the standard operating condition (Image 1), the system executes the "Bimanual Manipulation" action ($\mathcal{A}_2$). The behavior is characterized by two key metrics:

- **Velocity Tracking:** The graph shows that the actual end-effector velocities (Solid Blue/Red lines) closely match the "Desired Object Velocity" (Dashed Green line).
 - The *Desired Velocity* represents the feedforward reference generated by the **ObjectTask** (Priority 8/9), which computes the twist required to move the object to the goal.
 - The *Actual Velocity* confirms that the system successfully tracks this trajectory in the absence of safety conflicts.
- **Rigid Constraint Satisfaction:** A crucial indicator of bimanual stability is the distance between the two tool frames. The plots confirm this distance remains constant throughout the motion. This demonstrates the effectiveness of the **RigidConstrTask**, which is placed at **Priority 1** in the unified hierarchy. By strictly enforcing $v_L - v_R = 0$ in the relative frame, it ensures the object is neither dropped nor stressed, regardless of the motion commanded by lower-priority tasks.

2. Safety Altitude: Inactive vs. Active Comparison

When the goal altitude is set below the safety threshold (e.g., $z_{goal} = 0.1\text{m}$), the system exhibits a divergence between the "desired" intention and the "executed" safety behavior.

- **Inactive Safety (Desired Behavior - Green Line):** The dashed green line represents the reference velocity computed by the `ObjectTask`. Since the task is purely operational and unaware of the floor, it commands a negative vertical velocity ($v_z < 0$) to push the object down towards the unsafe goal.
- **Active Safety (Executed Behavior - Solid Lines):** The solid lines show the actual robot response. As the end-effectors breach the safety buffer, the `ToolAltitudeLimit` task activates.
 - **Priority Mechanism:** In the `main.m` unified list, `ToolAltitudeLimit` holds **Priority 4/5**, significantly higher than the `ObjectTask` (Priority 8/9).
 - **Outcome:** The safety task consumes the available degrees of freedom to nullify the vertical velocity ($v_z \approx 0$). The solid lines diverge from the green reference, effectively "clamping" the descent. The system hovers at the minimum safe altitude, ignoring the lower-priority command to descend further.

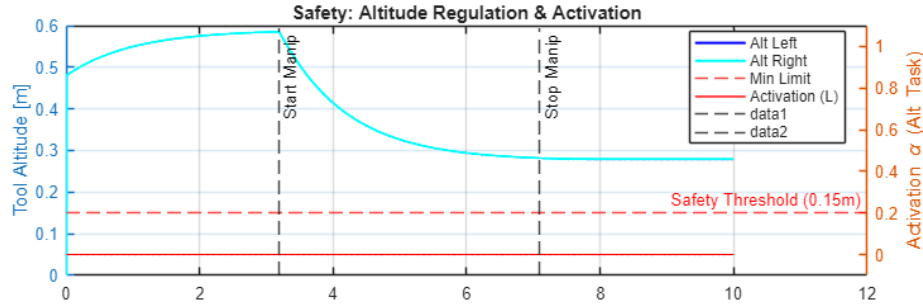


Figure 8: safety altitude not active

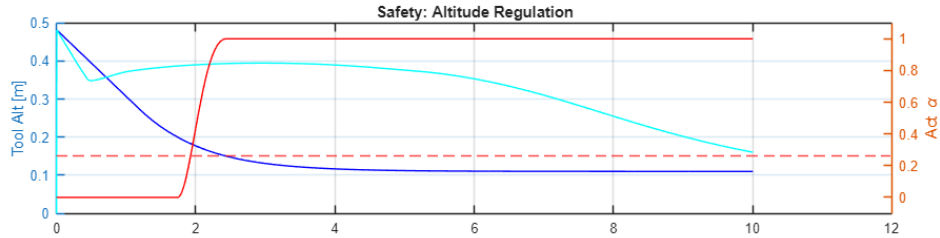


Figure 9: safety altitude active

3. Joint Limits Activation

In the scenario where the target is placed outside the reachable workspace (e.g., $x_{goal} = 10\text{m}$), the `JointLimitTask` intervenes to protect the manipulator hardware.

- **Behavioral Response:** As the arms extend and the joints approach their mechanical limits, the `JointLimitTask` (assigned **Priority 2/3**) generates a repulsive velocity in joint space.
- **Effect on Motion:** The graphs show the end-effector velocities smoothly decaying to zero (damping effect) even though the positioning error remains large. Because this safety task is higher in the hierarchy than the `ObjectTask`, it successfully overrides the motion command, preventing the robot from hitting the hard mechanical stops or entering a singular configuration.

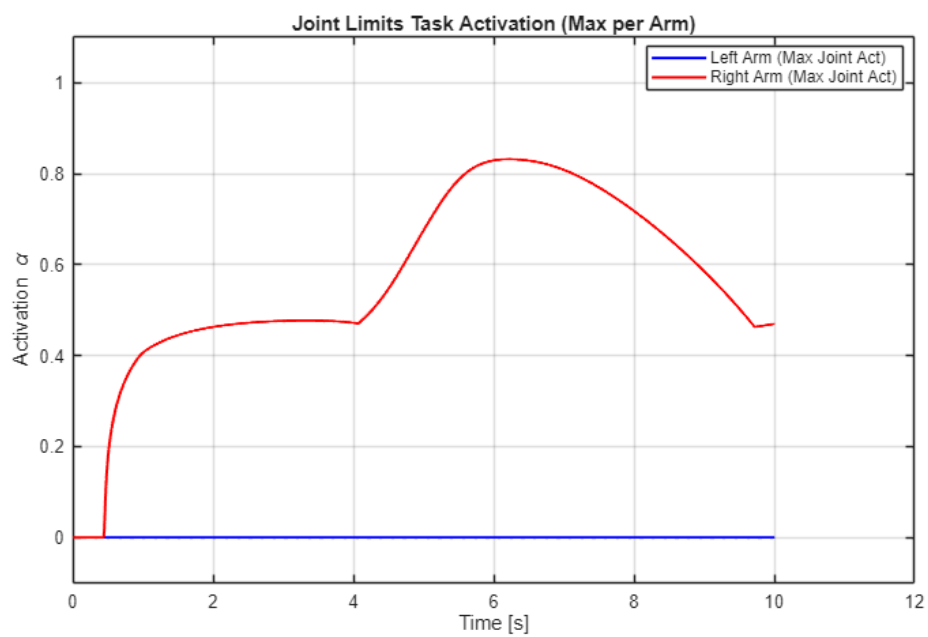


Figure 10: Joint Limits Activation

3 Exercise 3: Cooperative manipulation

In this exercise, it is required to perform cooperative manipulation by implementing the task priority algorithm considering the two Franka Panda manipulators as two distinct robots.

1. The first phase foresees to move the tool frames to the grasping points, by implementing the “move to” action for both manipulators. **Please note: each manipulator has his own Task Priority Inverse Kinematic Algorithm.** Define the goal frames for each manipulator such that their origin corresponds to the grasping points. **HINT:** the position of the grasping points can be computed by knowing the origin of the object frame wO_o and the object length l_{obj} . The goal orientation of the tool frames is obtained by rotating the tool frames 20 deg around their y-axis.

$${}^wO_o = [0.5, 0, 0.59]^\top (m); \quad (20)$$

$$l_{obj} = 6 \text{ (cm)}. \quad (21)$$

During this phase of the mission, the following safety tasks must be implemented: *end-effector minimum altitude* and *joint limits*. The joint limits specifications can be found in the datasheet of the robot. The minimum altitude from the table should be 15 cm.

2. Once the manipulators reach the grasping points the second phase of the mission should start. Implement the *Cooperative Rigid Constraint* task to carry the object as a rigid body.
 - (a) Define the object frame as a rigid body attached to the tool frame of each manipulator.
 - (b) Define the rigid grasp task.
 - (c) You have to move the object to another position while both manipulators hold it firmly. The desired object goal position is

$${}^wO_g = [0.60, 0.40, 0.48]^\top (m) \quad (22)$$

- (d) Compute the *non-cooperative* object frame velocities. **HINT:** Suppose manipulators communicate ideally and can exchange the respective end-effector velocities
- (e) Apply the coordination policy to the *non-cooperative* object frame velocities.
- (f) Compute the *cooperative* object frame velocities.

Note that the transition for the *Cooperative Rigid Constraint* should be a binary one, i.e., without smoothness. This is the nature of the constraint, i.e., either it exists or not. Modify the *ActionManager* class to take into account the different nature of this task (a constraint one).

3. Once the object has been moved to the required position you have to implement a final phase of the mission in which the joint velocities are set to zero, and every action is deactivated except for the minimum altitude task.

Again, test it twice, once with the provided (reachable) goal, and then with a goal or with a different grasp configuration that triggers the activation of multiple joint limits or a joint limit and minimum altitude by one manipulator. The idea is that this second position should trigger the cooperation policy (i.e., the cooperative velocity of one manipulator should be different than the original desired object velocity, and that the cooperative velocity is very similar to its non cooperative one, showing that the other robot adapts).

3.1 Q1: Report the unified hierarchy of tasks used and their priorities in each action. Report clearly which action is used by TPIK1 and TPIK2 in the cooperative algorithm.

In the cooperative manipulation architecture (Phase 2), the control problem is split into two distinct steps (TPIK1 and TPIK2) to implement the “Cooperative Task-Space Inverse Kinematics” algorithm. This separation allows the system to first compute a desired object velocity based on individual goals, and then strictly enforce coordination constraints during execution.

1. TPIK1 and TPIK2 Definition

- **TPIK1 (Non-Cooperative Computation):** This step corresponds to the *Compute* phase. It calculates the "desired" joint velocities ($\dot{q}_{nc}$) for each arm independently, as if they were performing the object motion task alone.
 - **Action Manager:** `actionManagerL` and `actionManagerR`.
 - **Action Name:** "Coop Calc" ($\mathcal{A}_{calc}$).
 - **Primary Task:** `ObjectTask`.
 - **Goal:** Provides the reference input to compute the coordination policy weights and the cooperative velocity vector $\mathbf{v}_{coop}$.
- **TPIK2 (Cooperative Execution):** This step corresponds to the *Execute* phase. It computes the final joint velocities ($\dot{q}$) sent to the robot. It enforces the rigid grasp constraints and tracks the globally computed cooperative velocity.
 - **Action Manager:** `actionManagerL_coop` and `actionManagerR_coop`.
 - **Action Name:** "Coop Exec" ($\mathcal{A}_{exec}$).
 - **Primary Task:** `CooperativeRigidConstrTask`.
 - **Goal:** Ensures physical consistency of the bimanual grasp while following the trajectory.

2. Unified Hierarchies of Tasks

The priorities differ between the Action Managers used for computation (Primary) and execution (Cooperative).

Primary Hierarchy (Used in Phase 1 & TPIK1)

Used for independent approach ($\mathcal{A}_1$) and non-cooperative computation ($\mathcal{A}_{calc}$).

Table 4: Task priorities for the Primary Action Manager. Priority 1 = Highest.

Task Name	Type	$\mathcal{A}_1$ (Appr.)	$\mathcal{A}_{calc}$ (TPIK1)	$\mathcal{A}_3$ (Stop)
<code>ToolAltitudeLimit</code>	I	1	1	1
<code>JointLimitTask</code>	I	2	2	
<code>tool_task</code>	E	3		
<code>ObjectTask</code>	E		3	
<code>StopTask</code>	E			2

Execution Hierarchy (Used in TPIK2)

Used strictly for the cooperative execution in Phase 2. Note that the constraint task is prioritized to ensure the object is not dropped.

Table 5: Task priorities for the Cooperative Execution Manager (TPIK2).

Task Name	Type	$\mathcal{A}_{exec}$ (TPIK2)
<code>CooperativeRigidConstrTask</code>	E	1
<code>ToolAltitudeLimit</code>	I	2
<code>JointLimitTask</code>	I	3

3.Task Descriptions

The following list describes the technical role of each task within the architecture:

- **ObjectTask (E):** Active during TPIK1. It computes the Jacobian and reference velocity required to move the object frame (wT_o) to a desired goal pose. It treats the arm as if it were solely responsible for the object’s motion, disregarding the other arm. The output from this task is used to estimate the ”intention” of the manipulator.
- **CooperativeRigidConstrTask (E):** The core task of TPIK2. It serves two purposes simultaneously: 1. **Constraint Enforcement:** It imposes that the relative velocity between the two end-effectors (projected on the object) is zero ($v_L - v_R = 0$), maintaining the rigid grasp. 2. **Tracking:** It tracks the cooperative velocity vector ($\mathbf{v}_{coop}$) computed by the weighted average policy in the main loop.
- **ToolTask (E):** Used in Phase 1 (Navigation/Approach). It controls the Cartesian pose of the end-effector to align it with the grasping point on the object. It allows independent motion of the arms before contact is made.
- **ToolAltitudeLimit (I):** A safety inequality task. It monitors the vertical position (z) of the end-effector. If the tool gets too close to the table (unsafe altitude), it generates a repulsive velocity to prevent collision. It is active in all phases to guarantee hardware safety.
- **JointLimitTask (I):** A safety inequality task that prevents the robot’s joints from reaching their mechanical limits. It uses a soft activation buffer to smoothly decelerate and repel the joints when they approach the boundaries of their workspace.
- **StopTask (E):** A regulation task active in Phase 3. It sets the desired velocity reference to zero for all joints, bringing the system to a halt safely while respecting higher-priority safety constraints (like altitude).

3.2 Q2: Comment the behaviour of the robots, using relevant plots. In particular, make sure there are differences between the desired object velocity and the non-cooperative Cartesian velocities at least in one simulation. Show also how the cooperative velocities of the two end-effectors behave. Add plots to show that the cooperation unfolds effectively (e.g., the distance between the tool frames is constant to show that the kinematic constraint is satisfied)

The simulation implements a bimanual manipulation mission governed by a Task-Priority Inverse Kinematics (TPIK) framework. The behavior evolves from decentralized approaching to tightly coupled cooperative manipulation, while strictly enforcing safety boundaries.

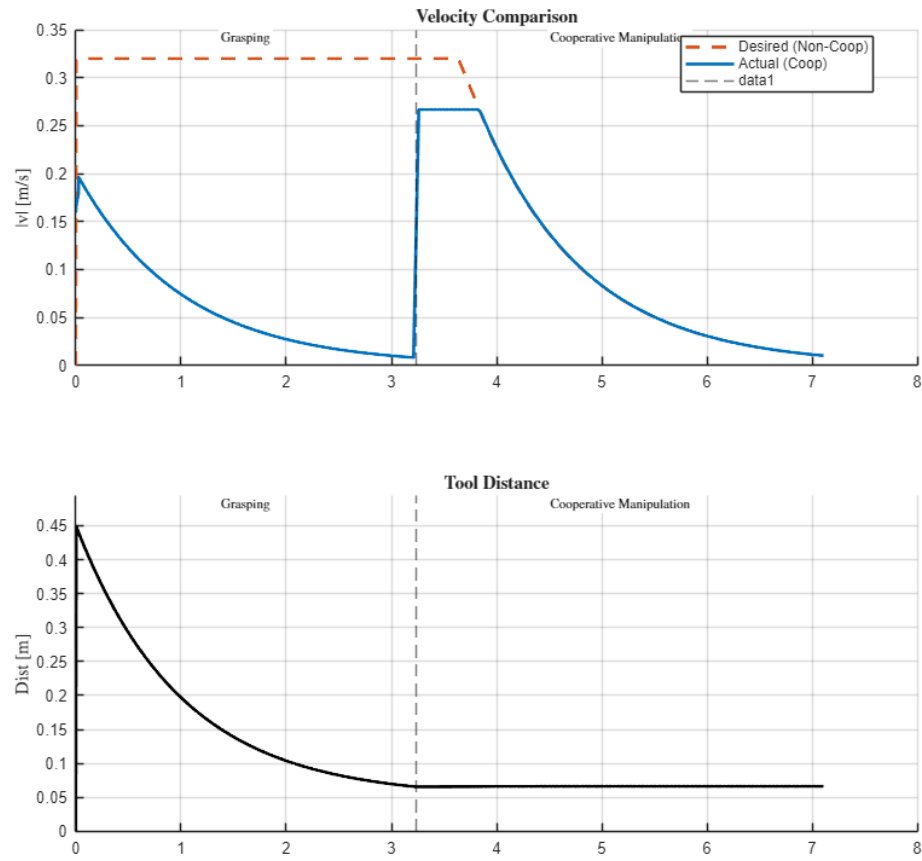
3.3 Mission Execution: Transition from Independence to Cooperation

The mission profile is characterized by a fundamental switch in control strategy between Phase 1 and Phase 2, which is clearly observable in the velocity and error plots.

- **Decentralized Grasping (Phase 1):** Initially, the two Panda robots operate independently. The velocity profiles in this phase are driven solely by the `tool_task`, aiming to minimize the error between the end-effectors and their respective grasping points defined in `main.m`. The motion is unconstrained by the partner robot, resulting in purely error-driven kinematics.
- **Rigidly Constrained Cooperation (Phase 2):** Upon grasping, the control architecture enforces a *Rigid Constraint*. The system computes a ”consensus velocity” ($\dot{x}_{hat}$) by averaging the individual goal-directed velocities. This velocity is then projected onto the admissible motion subspace of the rigid grasp using the `CooperativeRigidConstrTask`.

The relevant plots for this phase demonstrate that the **Actual Velocity** of the robots tracks the **Cooperative Reference** ($\bar{\dot{x}}_{coop}$), which satisfies the kinematic constraint $v_{left} = v_{right}$

(expressed in the object frame). This ensures that no internal stresses are applied to the object, even if the individual non-cooperative references diverge.



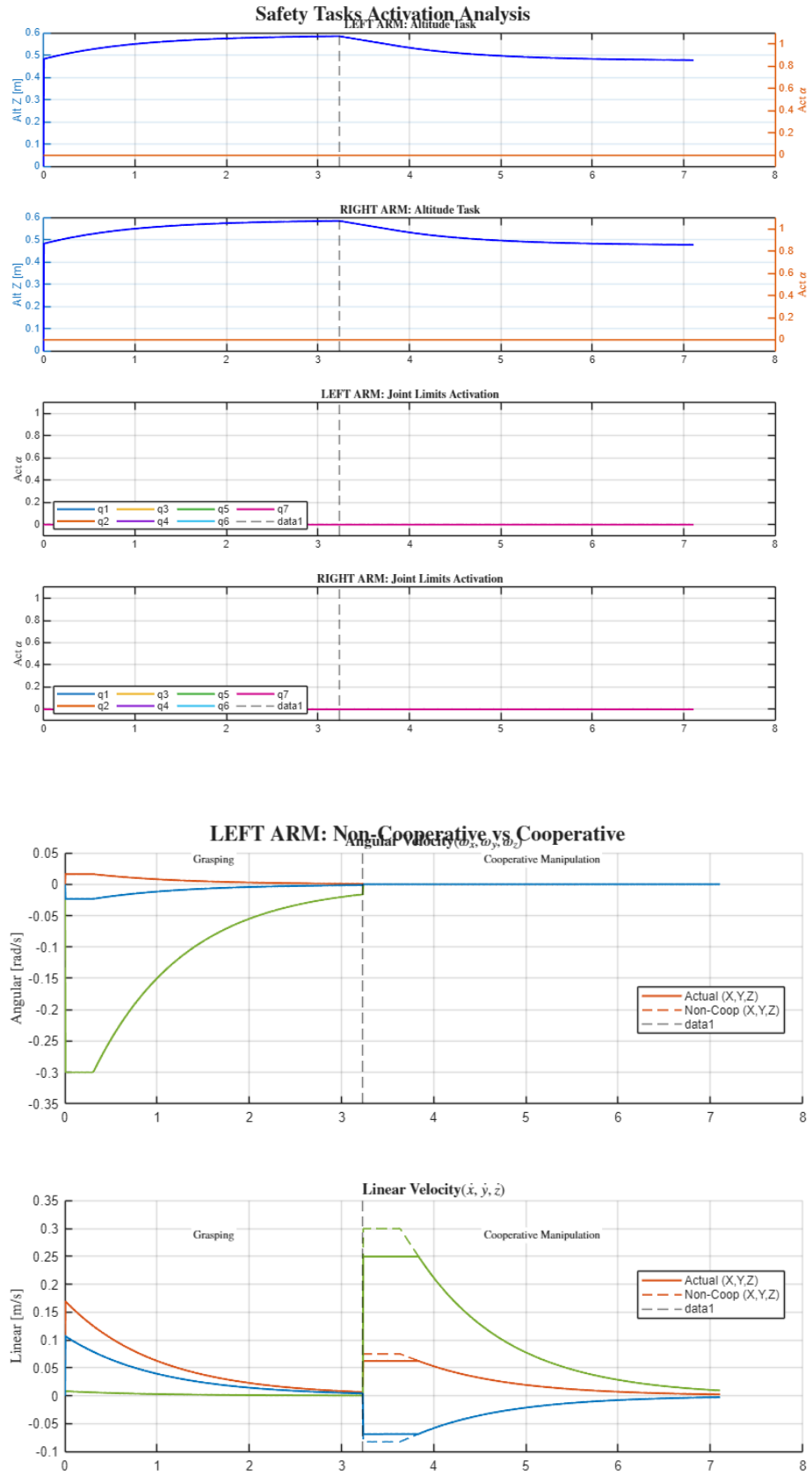
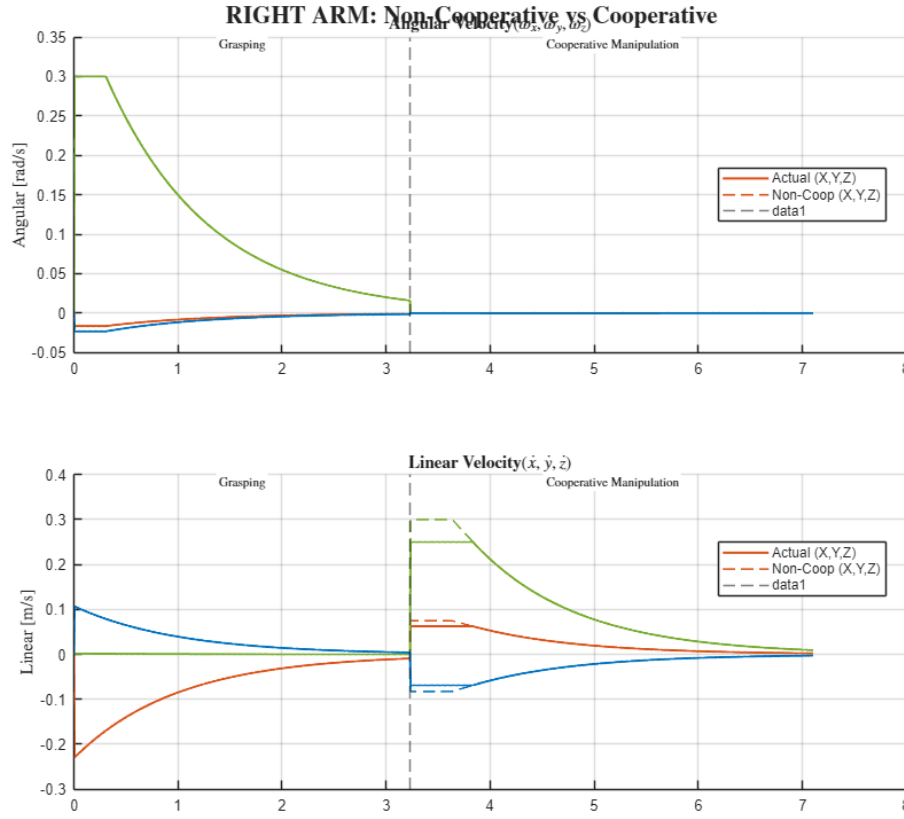


Figure 11:



4 Analysis of Unreachable Goal and Joint Limit Safety

In this scenario, the cooperative goal was purposely set to a position $\mathbf{p}_g = [0.85, 0, 0.07]^T$ m, which lies near or beyond the boundary of the bimanual workspace. The simulation results highlight the intervention of the `JointLimitTask` in preserving the robot's integrity.

4.1 Task Activation and Conflict

As the robots transport the object towards the distant goal, their kinematic configuration approaches the mechanical limits of the joints (typically the extension joints).

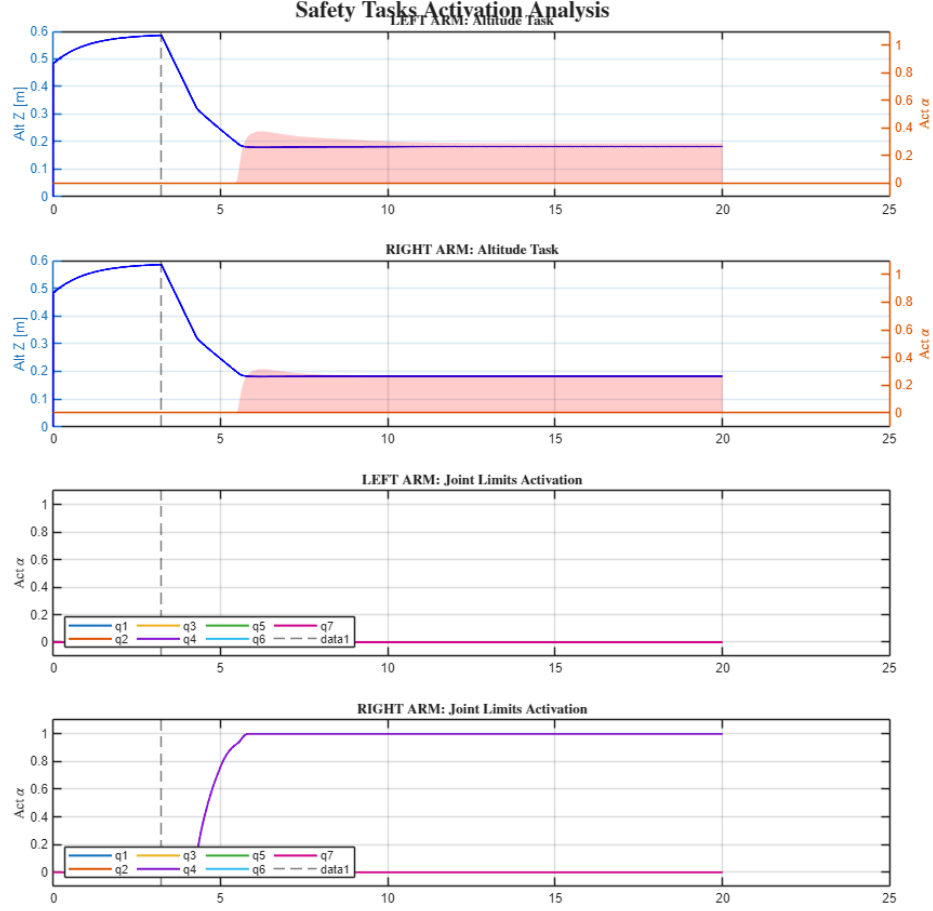
- **Activation:** The `JointLimitTask` monitors the distance of each joint q_i from its limits (q_{min}, q_{max}). As seen in the *Safety Tasks Analysis* plot, the activation function α_{JL} for specific joints rises to 1 as they enter the safety buffer zone defined by δ .
- **Velocity Suppression:** The activated safety task generates a repulsive velocity reference $\dot{q}_{ref}$ to prevent further extension. Since the safety tasks are inserted with high priority in the `ActionManager`, this repulsive action nullifies the motion commands generated by the lower-priority `CooperativeRigidConstrTask`.

4.2 Resulting Behavior

The interaction results in a safe "clamping" of the system:

1. **Motion Arrest:** The robots stop moving towards the goal before reaching it. The Cartesian Velocity plots show the actual velocity dropping to zero, deviating from the non-zero reference velocity requested by the navigation task.
2. **Steady-State Error:** A permanent position error persists between the object frame and the goal frame, confirming that the target is kinematically unreachable under the active safety constraints.

3. **Safety Guarantee:** Crucially, the joint positions remain strictly bounded within $[q_{min}, q_{max}]$, validating the robustness of the hierarchical control architecture.



5 Cooperative Accommodation and Joint Limit Analysis

In this scenario, the cooperative goal is set to $\mathbf{p}_g = [0.85, 0, 0.15]^T$. This position represents a critical workspace boundary: it requires near-maximum extension ($x = 0.85$ m) while simultaneously grazing the vertical safety limit ($z = 0.15$ m). The simulation highlights the system's ability to handle unilateral constraints through cooperation.

5.1 Cooperative Accommodation of Blocked Agents

A key feature of the implemented control architecture is the "accommodation" behavior. The plots demonstrate that when one agent (e.g., the Right Arm) is blocked by its internal safety constraints (Joint Limits), the partner agent (Left Arm) autonomously adapts its motion to preserve the grasp integrity.

- **Unilateral Constraint:** As the system approaches the goal, the Right Arm reaches its kinematic extension limit first. The `JointLimitTask` suppresses its non-cooperative velocity command ($\dot{q}_{r,nc} \rightarrow 0$) to prevent violation.
- **Consensus Propagation:** This zero-velocity intent propagates to the cooperative layer. The consensus velocity $\dot{x}_{hat}$ drops significantly because one of the two contributing terms has become zero:

$$\dot{x}_{hat} = \frac{\mu_l \dot{x}_{l,nc} + \mu_r \cdot \mathbf{0}}{\mu_l + \mu_r} \quad (23)$$

- **Accommodating Behavior:** The `CooperativeRigidConstrTask` projects this reduced velocity onto the rigid constraint manifold. Consequently, the Left Arm receives a reference $\bar{x}_{l,coop}$ that commands it to stop, effectively "waiting" for the blocked partner.
- **Visual Evidence:** The velocity plots show the Left Arm's *Actual* velocity dropping to zero in sync with the Right Arm, even though the Left Arm's *Non-Cooperative* task would still drive it forward. This proves the validity of the rigid coupling.

5.2 Joint Limit Activation Function

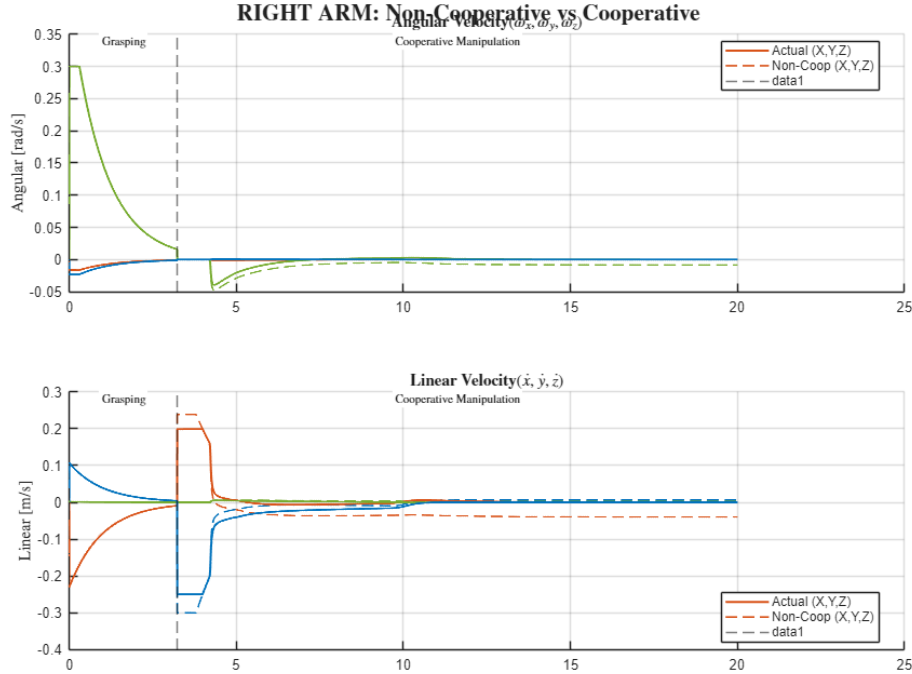
The behavior described above is triggered by the activation of the `JointLimitTask`. The activation profile for the critical joints (typically the elbow joint q_4 or shoulder joints involved in extension) provides insight into the safety mechanism.

- **Activation Logic:** The task monitors the distance $d(q)$ to the joint limits. The activation function is defined as:

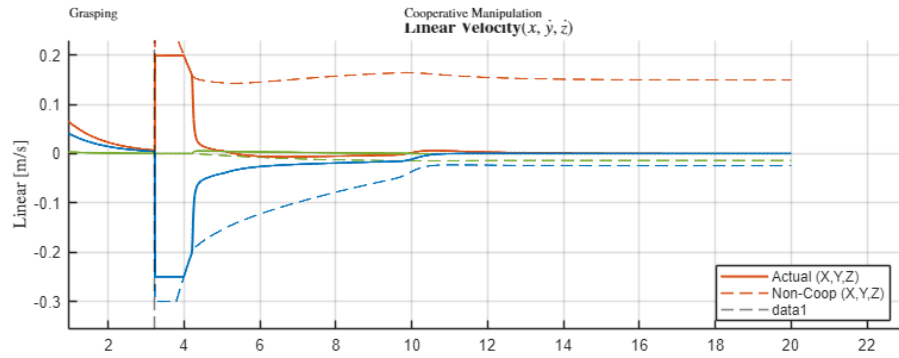
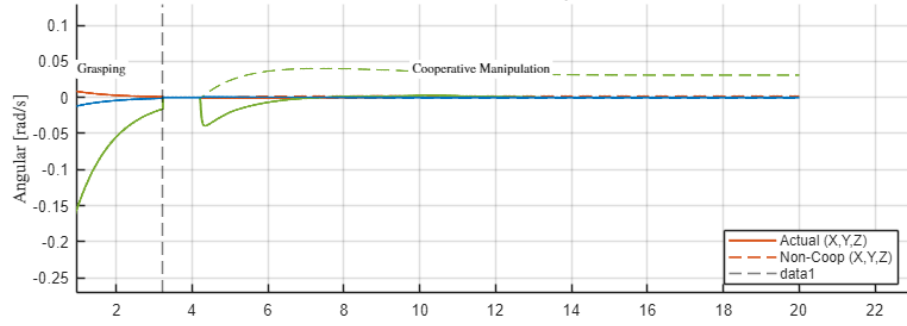
$$A_{ii} = \text{DecreasingBellShaped}(0, \delta, 0, 1, d_{min}(q_i)) \quad (24)$$

where $\delta = 0.2$ rad is the safety buffer.

- **Behavior at Boundary ($x = 0.85$):** The activation plot shows the signal α_{JL} rising sharply to 1.0 for the extensor joints as the tool reaches the radial distance of 0.85 m.
- **Repulsive Action:** Unlike the altitude task which creates a floor, the Joint Limit activation acts as a "soft wall" in the joint space. The plots confirm that as $\alpha_{JL} \rightarrow 1$, the corresponding joint velocities are clamped, preventing the robot from entering a singular configuration or hitting the mechanical hard stop.



LEFT ARM: Non-Cooperative vs Cooperative



Safety Tasks Activation Analysis

